

Solo Developer Build Guide

AI Growth Team SaaS — Scratch se Launch tak, Step by Step

Har step pe clear hai: kya **Claude Code** karega, kya **aap khud** karenge

Spec v3.0 ka companion guide • Beginner-friendly Hinglish • August 2026

Ye Guide Kaise Use Karein

Ye guide aap ke Spec v3.0 ko **actual kaam** mein badalti hai. Har step ek chhota, complete kaam hai. Ek step khatam karo, checkpoint pass karo, phir agla. Steps skip mat karna — har step agle ki foundation hai.

Box ka color	Matlab
 CLAUDE CODE	Ye kaam AI karega. Dark box mein jo prompt likha hai, usko copy karke Claude Code mein paste karo . Claude Code files banayega, code likhega, errors fix karega. Aap sirf review karke "yes" karte jao.
 AAP KHUD KARENGE	Ye manual kaam hai — account banana, API key copy karna, button click karna, test karna. AI ye aap ke liye nahi kar sakta. Exact clicks likhe hain.
 CHECKPOINT	Step tabhi complete hai jab ye cheez kaam kare. Nahi karti? Claude Code ko error paste karo: " <i>ye error aa raha hai, fix karo</i> ".

SAWAL: Pehle kya shuru karun?

Aaj hi, isi order mein: **(1)** Step 0 ke saare accounts banao (2–3 ghante ka boring kaam, lekin ek hi baar) → **(2)** Step 0.4 ki API applications file karo (ye hafton mein approve hoti hain, isliye Day 1 pe) → **(3)** Step 1 chalao — aaj raat tak aap ki app live hogi. Bas. Kal Step 2.

Golden Rules (poore project ke liye)

- **Ek waqt mein ek step.** Claude Code ko poora project ek prompt mein mat do — wo confuse hoga aur aap review nahi kar paoge.
- **Har step ke baad commit.** Har prompt ke end mein Claude Code khud `git commit` karega (prompts mein likha hai). Kuch toot jaye to wapis ja sakte ho.
- **Samajh na aye to poochho.** Claude Code se kaho: "*ye file kya karti hai, simple words mein samjhao*". Beginner ka sab se bara advantage yehi hai.
- **API keys sirf `.env` mein.** Kabhi code file mein paste nahi, kabhi git mein commit nahi. Claude Code isko khud handle karega, lekin aap bhi dhyan rakho.
- **Error aaye to poora error copy karke Claude Code ko do.** Screenshot se behtar text hai.

STEP 0 — Setup Day AAJ (1 din)

Aaj sirf accounts aur installs. Coding kal se. Sab manual hai — chai leke baith jao.

0.1 — Machine setup

30 min

 **AAP KHUD:**

1. **Node.js** install karo: nodejs.org se LTS version download → install → terminal mein `node -v` chala ke confirm.

2. **Git** install karo: git-scm.com (Windows pe "Git Bash" bhi mil jayega — wohi terminal use karna).

3. **Claude Code** install: terminal mein `npm install -g @anthropic-ai/claude-code` → phir `claude` chala ke login karo.

4. **VS Code** install karo (code dekhne/edit ke liye): code.visualstudio.com

0.2 — Free accounts (sab pe sign up)

60-90 min

 **AAP KHUD — har account banao aur API keys ek private note mein save karte jao:**

Service	Kis liye	Kya save karna hai
github.com	Code rakhne ke liye	Username
vercel.com	Frontend hosting (GitHub se login)	—
supabase.com	Database + auth + storage	Project URL, anon key, service_role key (Step 2 pe)
upstash.com	Redis (job queue ke liye)	Redis URL + token (Step 7 pe)
railway.app	Agent server hosting	—
build.nvidia.com	Nemotron Lightning API (NIM)	API key
platform.deepseek.com ya aistudio.google.com	Article writing model	API key (~\$5 credit dal do)
dataforseo.com	Keywords + SERP data	Login + password (API creds), trial credit
resend.com	Notification emails	API key (Step 17 pe)

Kharcha abhi: practically zero — sab free tiers/trials hain. Sirf DeepSeek/Gemini mein \$5 credit dalna parega jab Writer test karoge.

0.3 — Test WordPress site

20 min

 **AAP KHUD:**

1. Ek test WP site chahiye jisme pipeline articles publish karegi. Sab se asaan: apni kisi purani/spare WordPress site use karo, ya InfinityFree/kisi hosting pe ek throwaway WP install karo.

2. WP Admin → Users → apna user → neeche **"Application Passwords"** → naam do "saas-test" → Generate → password copy karke note mein save karo.

0.4 — Lambi approvals AAJ file karo

45 min

 **AAP KHUD:**

1. **Google Cloud:** console.cloud.google.com → naya project → enable karo: "Search Console API", "Google Analytics Data API", "My Business" APIs → OAuth consent screen setup shuru karo (verification baad mein complete hogi).

2. **Meta:** developers.facebook.com → naya app (Business type) → Instagram/Pages products add karo. Business verification ka process note kar lo.

3. **LinkedIn:** developer.linkedin.com → app banao → Community Management API ka access request file karo.

Note: In teeno ki approval mein aap ko rejection bhi aa sakta hai pehli baar — normal hai, requirements पूरी karke dobara file karna. Isi liye Day 1 pe shuru.

Step 1 — Project scaffold + hosting

~2 ghante

AAP KHUD (pehle): Ek folder banao (jaise `growth-saas`), terminal us folder mein kholo, `claude` type karke Claude Code shuru karo.

CLAUDE CODE — ye prompt paste karo:

Create a new Next.js 14 (App Router) project in this folder called "growth-saas" with TypeScript, Tailwind CSS, and shadcn/ui set up. Structure it as the frontend for a multi-tenant SaaS dashboard. Add: a clean folder structure (app/, components/, lib/), a .env.example listing SUPABASE_URL, SUPABASE_ANON_KEY, SUPABASE_SERVICE_ROLE_KEY placeholders, a .gitignore that excludes .env files, and a simple landing page that says "AI Growth Team" with a login button linking to /login (page can be empty for now). Initialize git and make the first commit. Then explain to me in simple terms what each top-level folder does.

AAP KHUD (baad mein):

- GitHub pe naya repo banao (private) → Claude Code se kaho: *"push this repo to [repo URL]"*.
- vercel.com → Add New Project → apna GitHub repo select → Deploy. 2 minute mein live URL milega.

CHECKPOINT: Vercel ka URL browser mein kholo — "AI Growth Team" landing page dikhni chahiye.

Step 2 — Database + tenant security (RLS)

~3 ghante

AAP KHUD (pehle):

- supabase.com → New Project (region: Mumbai/Singapore) → strong DB password save karo.
- Project Settings → API → **URL, anon key, service_role key** copy karo.
- SQL Editor → chalao: `create extension if not exists vector;` (ye pgvector on karta hai).
- Project folder mein `.env.local` file banao aur teeno values paste karo (Claude Code se format poochh lena).

CLAUDE CODE:

Set up Supabase for this project. Install @supabase/supabase-js and @supabase/ssr, create the client helpers in lib/. Then write a SQL migration file (supabase/migrations/001_init.sql) for a multi-tenant SaaS with these tables: tenants (company profile: name, website_url, niche, tone_profile jsonb, icp_profile jsonb), memberships (user_id, tenant_id, role), integrations (tenant_id, type, encrypted_credentials jsonb, status), content_items (tenant_id, type article/social/gbp, status draft/awaiting_approval/approved/published/failed, title, body, blueprint jsonb, meta jsonb), site_pages (tenant_id, url, title, content_text, embedding vector(768)), jobs_log (tenant_id, agent, action, status, detail jsonb), leads (tenant_id, name, company, email, phone, source, icp score, reason, stage), notifications (tenant_id, user_id, type, payload jsonb, read). Every table gets Row Level Security policies so users only access rows of tenants they belong to (via memberships). Explain the RLS concept to me simply after writing it. Commit when done.

AAP KHUD: Migration ka SQL Supabase ke SQL Editor mein paste karke Run karo. (Claude Code se kaho: *"give me the final SQL to paste in Supabase SQL editor"*.)

CHECKPOINT: Supabase → Table Editor mein saari tables dikh rahi hain, aur har table pe "RLS enabled" ka green tag hai.

Step 3 — Login/Signup + dashboard shell

~3 ghante

CLAUDE CODE:

Build authentication with Supabase Auth: /login and /signup pages (email+password and Google OAuth button), email verification flow, and a protected /app route group. On first login, create a tenant row and membership for the user. Build the dashboard shell at /app: left sidebar (Dashboard, Content, Approvals, Leads, Settings), a topbar showing tenant name, and an empty dashboard page with placeholder agent status cards for: Boss AI, Keyword Finder, Writer, Social Media, SEO Analyzer, Lead Hunter — each card shows name, status dot (working/idle/offline), and current task text (static data for now). Mobile responsive. Commit.

AAP KHUD: Supabase → Authentication → Providers → Google enable karo (Google Cloud wale project se OAuth client ID bana ke — Claude Code se step-by-step poochh lena: *"guide me through creating the Google OAuth client for Supabase"*).

CHECKPOINT: Vercel URL pe signup karo → email verify → login → dashboard mein 6 agent cards dikhein.

Step 4 — Onboarding wizard

~4 ghante

📄

CLAUDE CODE:

Build a multi-step onboarding wizard at /app/onboarding shown to new tenants: Step A company profile (what the business sells, target customers, service area), Step B website URL input, Step C tone questionnaire (formal/casual slider, topics to avoid, audience description), Step D publishing frequency (articles per week, preferred days), Step E connect WordPress (site URL + username + application password fields, store encrypted in integrations table, add a "Test connection" button that calls the WP REST API /users/me endpoint). Save everything to the tenants and integrations tables. Show a completion screen. Commit.

✓

CHECKPOINT: Wizard complete karo apni test WP site ke saath — "Test connection" green ho.

Step 5 — Crawler + embeddings (niche map)

~4 ghante

📄

CLAUDE CODE:

Create a site crawler as a Next.js API route (or edge-safe server action) that runs after onboarding: fetch the tenant's site pages (sitemap.xml first, fallback to homepage link crawl, cap 100 pages for now), extract readable text per page, generate embeddings for each page using a free/open embedding model API (use a provider-agnostic adapter in lib/ai/embeddings.ts so we can swap providers), and store in site_pages with pgvector. Add a "niche summary" step: send page titles+excerpts to an LLM adapter (lib/ai/llm.ts, provider-agnostic, reading keys from env) and save a niche + topics summary into tenants.tone_profile. Show crawl progress in the onboarding completion screen. Commit.

Embedding model kaunsa? Claude Code se poochho: "which free embedding API should I use here, give me options with pros/cons" — aur jo wo suggest kare uski key .env mein dal do. Adapter pattern ki wajah se baad mein badalna 1 line ka kaam hoga.

✓

CHECKPOINT: Supabase → site_pages table mein aap ki test site ke pages embeddings ke saath dikhein.

Step 6 — Agent server + job queue

~4 ghante

 **AAP KHUD (pehle):** upstash.com → Create Redis Database (region same as Supabase) → connection URL copy.

 CLAUDE CODE:

Create a second app in this repo under /agent-server: a Node.js + TypeScript Express server with BullMQ connected to Upstash Redis (REDIS_URL from env). Set up: a job queue per agent type (keyword, writer, social, seo, leads), a worker framework where each agent is a class with a run(job) method, a jobs_log writer that records every job start/finish/error to Supabase (service role key), hard per-tenant daily caps read from a config, and retry with exponential backoff (3 attempts) then mark failed. Add a /health endpoint. Also add Socket.io that emits agent status events (agent, tenant, status, task) – the frontend will subscribe later. Commit.

 **AAP KHUD (baad):** railway.app → New Project → Deploy from GitHub repo → root directory /agent-server set karo → saare env variables paste karo (Claude Code se list maang lo: "list every env var the agent server needs") → Deploy. Free trial ke baad ~\$5/month ka Hobby plan lena hoga — ye wohi "always-on" cost hai.

 **CHECKPOINT:** Railway ka URL + /health browser mein "ok" de.

Step 7 — Lightning (Boss AI ka dimagh)

~3 ghante

 CLAUDE CODE:

In /agent-server, create lib/lightning.ts: a client for NVIDIA NIM's OpenAI-compatible API (key NVIDIA_API_KEY from env) using the Nemotron 3.5 Lightning model, with helpers for: classify(text, labels), summarize(text), and generateJson(prompt, schema) with strict JSON parsing + one retry on invalid JSON. Then create the Boss AI orchestrator: a function that takes a tenant task request (e.g. "new article cycle"), breaks it into queued jobs in the right order, logs the plan to jobs_log, and emits Socket.io status updates as jobs progress. Write a small test script I can run with npm run test:boss that fires a dummy cycle and prints the flow. Commit.

 **AAP KHUD:** build.nvidia.com se API key .env mein dalo. Test script chalao: npm run test:boss .

 **CHECKPOINT:** Test script console mein Boss AI ka plan aur Lightning ke responses print kare.

Step 8 — Live status dashboard se jodo

~2 ghante

 CLAUDE CODE:

Connect the frontend dashboard's agent status cards to the agent-server's Socket.io feed (URL from env). Cards should live-update status dot and current task text from events, falling back to "idle" when no events. Add a small activity feed panel on the dashboard showing the last 10 jobs_log entries for the tenant. Commit.

 **CHECKPOINT:** Test script dobara chalao — dashboard ke cards LIVE change hote dikhein. (Ye moment enjoy karna — aap ki AI team ki pehli saans hai. 🎉)

Step 9 — Keyword Finder agent

~4 ghante

📋 CLAUDE CODE:

Build the Keyword Finder agent in /agent-server: given a tenant, (1) call DataForSEO (login/password from env) keyword ideas + search volume + difficulty for the tenant's niche topics (from tone_profile), (2) query site_pages embeddings to exclude topics already covered, (3) score candidates with Lightning (volume x low difficulty x niche relevance x not-covered) using generateJson, (4) save top 5 to content_items as type "topic_suggestion" with score+reason in meta, (5) emit status events during the run. Add a "Topics" section on the dashboard Content page listing suggestions with Approve buttons (approval just updates status for now). Commit.

🔗 AAP KHUD: DataForSEO dashboard se API login/password .env mein; trial credits check karo.

✅ CHECKPOINT: Dashboard pe "Run Keyword Finder" (Claude Code se ek trigger button banwa lo) → 5 topics reasons ke saath aa jayein.

Step 10 — SERP analyzer + Blueprint

~5 ghante

📋 CLAUDE CODE:

Build the SERP analysis stage: when a topic is approved, (1) fetch top 10 organic results for the keyword via DataForSEO SERP API, (2) fetch each result URL and extract readable content (use a readability library, skip failures gracefully), (3) with Lightning, analyze each competitor: heading structure, word count, topics covered, gaps, (4) fetch People Also Ask + related searches from the SERP response, (5) generate a blueprint with Lightning generateJson: 3 title options, full H2/H3 outline covering union of competitor topics + gaps + PAA, target word count above top-10 average, and 3-5 internal link candidates from site_pages by embedding similarity. Save blueprint into the content_item. Show the blueprint in the dashboard with an "Approve blueprint / Auto-continue" setting per tenant. Commit.

✅ CHECKPOINT: Approved topic ka blueprint dashboard pe dikhe — titles, poora outline, target length.

Step 11 — Writer agent (frontier model)

~4 ghante

📋 CLAUDE CODE:

Build the Writer agent: extend lib/ai/llm.ts into a model adapter with named providers (deepseek, gemini, anthropic) selected by env WRITER_PROVIDER. The Writer takes an approved blueprint and writes the article section-by-section (one call per H2 group to stay reliable), injecting the tenant's tone_profile and inserting the internal links naturally. Assemble final HTML, save to the content_item body, status "awaiting_qc". Track token cost per article in meta. Commit.

🔗 AAP KHUD — YE STEP IMPORTANT HAI: Model quality test aap ka kaam hai, AI ka nahi:

- Ek hi blueprint pe 2-3 providers se article likhwao (env badal badal ke).
- Teeno ko khud parho — kaunsa aap ki site pe publish karne layak hai?
- Jo jeeta, wohi default. Ye faisla aap ke product ki quality decide karta hai.

✅ CHECKPOINT: Ek complete article (1500+ words, headings, internal links) content_item mein saved ho.

Step 12 — Quality gate + Approval card + Publish

~5 ghante

📋 CLAUDE CODE:

Build (1) the quality gate that runs after writing: keyword present in title/H1/intro check, all blueprint H2s present, word count vs target, every link resolves (HTTP HEAD), tone similarity vs tone_profile via embeddings, flag dated claims/statistics; write results into meta.qc and set status "awaiting_approval". (2) The Approvals page: card per item showing title, article preview, QC results as pass/warn badges, and Approve / Edit (simple rich text) / Reject buttons. (3) On Approve: publish to the tenant's WordPress via REST API (posts endpoint, application password auth), save the live URL, status "published"; on failure use the retry policy and set "failed" with the error visible on the card. Commit.

✅ CHECKPOINT — PHASE 1 KA DIL: Topic approve → blueprint → article → QC → approval card → Approve click → **article aap ki test WP site pe LIVE**. Ye chal gaya to aap ke paas product hai.

Step 13 — Content calendar (scheduler)

~3 ghante

📋 CLAUDE CODE:

Build the scheduler: BullMQ repeatable jobs per tenant based on their onboarding frequency (e.g. Mon+Thu 9am) that auto-run the topic cycle. Add a Calendar page in the dashboard showing planned/in-progress/awaiting-approval/published items on a week grid, with pause/resume tenant automation toggle (vacation mode). Commit.

✔ CHECKPOINT:

Schedule set karo 5 minute baad ka (test) — cycle apne aap chale aur approval card aa jaye.

Step 14 — Notifications (PWA + push + email)

~4 ghante

📋 CLAUDE CODE:

Make the app an installable PWA (manifest, service worker) and add Web Push via FCM: when a content_item reaches awaiting_approval and the user has no active socket connection, send a push "Draft ready: [title]" that deep-links to the approval card. Add email fallback via Resend (RESEND_API_KEY): every approval request also sends an email with a magic link to the card. Add per-user notification preferences (push on/off, quiet hours). Guide me through creating the Firebase project and getting the FCM keys step by step before writing the code. Commit.

🔔 AAP KHUD:

(1) Firebase project + FCM keys — Claude Code aap ko clicks batayega. (2) resend.com API key. (3) Apne phone pe test: site kholo → "Add to Home Screen" → app band karke ek approval trigger karo → notification aani chahiye. iPhone pe home-screen install ke baad hi push chalti hai — dono test karna.

✔ CHECKPOINT:

Phone band-screen pe notification aaye, tap karo, seedha approval card khule.

-  **AAP KHUD:**
1. Apni real site (travel site perfect hai) ko tenant bana ke onboard karo, hafte ke 2-3 articles ka schedule set karo.

2. Har article ko approve karne se pehle khud parho — jo galat lage uski list banao.

3. Wo list Claude Code ko do: "*Writer prompt improve karo — ye problems aa rahi hain: [list]*". Prompt tuning hi asli product-making hai.

4. Google Search Console mein articles ki indexing dekhna shuru karo.

 **AAP KHUD — SALES (sab se important manual kaam):** Apne contacts mein se 3-5 chhoti companies/agencies se baat karo. Demo dikhao (isometric prototype + live pipeline). Pehle client ko 50% discount do, feedback lo. **Billing system ka wait mat karo — pehle 1-2 clients ko manual invoice bhejo.** Paying client ke bina Phase 2 shuru mat karna.

 **PHASE 1 COMPLETE JAB:** (1) aap ki site pe 5+ AI articles live hain jo aap khud publish-quality maante ho, (2) ek bahar ka insaan (pilot client) system use kar raha hai, (3) scheduler bina aap ke chhue cycle chala raha hai.

PHASE 2 — Distribution (Weeks 6-14, overlap ok)

Ab granularity kam — pattern aap seekh chuke ho: har feature = 1 Claude Code prompt + manual keys/testing. Order yehi rakho:

Feature	🛠️ Claude Code prompt ka core	👤 Aap ka manual kaam
Chatbot widget	"Build an embeddable JS chat widget served from /widget.js that answers via RAG over the tenant's site_pages embeddings using Lightning, captures name/email/phone into leads table as source=inbound, themed by tenant colors, with a signed site key per tenant"	Apni site pe script tag laga ke test; hot-lead notification check
Images agent	"Add an image step: generate/fetch a licensed image per article (adapter for Stability/Unsplash APIs), AI alt-text, WebP compress, upload to WP media"	Stability/Unsplash keys
Social auto-post	"Build the Social agent: per-platform caption generation with Lightning, scheduled posting via Meta Graph API / X API / LinkedIn API with per-platform adapters and token refresh"	Meta app review complete karna, X API tier kharidna, LinkedIn approval — ye 80% manual/waiting hai
Quora/Reddit draft-assist	"Build draft-assist: find relevant Quora questions and subreddits for a published article, draft platform-appropriate answers/posts, show in a Ready-to-Paste queue with copy buttons"	Drafts ki quality khud check karo, subreddit rules spot-check
GBP management	"Integrate Google Business Profile API: weekly post generation+publish, Q&A draft answers, connect flow in integrations"	GBP API access request (Google form) — pehle se file karo
Daily audit + uptime	"Build the SEO agent's daily audit job: broken links, missing meta, PageSpeed API scores, mobile issues; plus 5-min uptime pings and SSL expiry checks with instant alerts"	PageSpeed API key (free)
Isometric office UI	"Integrate the isometric office prototype (file attached) as the dashboard hero, driven by the existing Socket.io events instead of demo buttons"	Artist ko layered art commission karna (~\$300-800) — prototype hi brief hai

PHASE 3 — Growth Module (Leads) + Billing

Feature	📋 Claude Code prompt ka core	👤 Aap ka manual kaam
ICP interview	"Add an ICP interview step to onboarding: Boss AI asks who the ideal customer is (industry, size, location, deal size, disqualifiers) conversationally, saves structured icp_profile"	Apne pilot client ke saath khud chala ke dekho
Lead Hunter	"Build the Lead Hunter agent: search Google Places API + business directories for ICP matches, enrich with an email-finder API adapter (Hunter-style), verify emails, score each lead vs icp_profile with Lightning including a one-line reason, save daily batches to leads table"	Google Places API key (billing enable — free credit), Hunter/Apollo account
Mini-CRM	"Build the Leads page as a pipeline board: New/Contacted/Replied/Won/Lost columns, drag to move, lead detail drawer with activity log, stale-lead reminders"	—
Email outreach	"Build compliant outreach: sequences (first-touch personalized via writer model, 2 follow-ups), sent through a dedicated sending domain via SMTP adapter, hard caps 40/day/domain, mandatory opt-out link + suppression list, reply detection via IMAP/webhook that stops the sequence and notifies"	Bohat manual: alag sending domain kharido (client ki main domain KABHI nahi), 2–3 hafte warmup service chalaao, SPF/DKIM/DMARC set karo (Claude Code clicks batayega), pehli campaign khud approve karo
SMS/WhatsApp draft queue	"Build the Ready-to-Send queue: per-lead personalized SMS/WhatsApp drafts with tap-to-open sms:/wa.me deep links prefilled"	Khud apne phone se pehle 10 bhejo, tone check karo
Reviews + decay + digest + monthly report	Har ek ka pattern same: "Build [X] agent per spec section [Y]" — spec v3.0 file Claude Code ko de do context ke liye	GBP/Meta review scopes approvals
Billing	"Integrate Paddle (ya Lemon Squeezy): checkout, webhooks to subscription state, plan gates on features, usage metering on articles+outreach"	Paddle account + seller approval (KYC docs) — 1–2 hafte, pehle se file karo

Aakhri baat

Ye poora project 15-minute ke chhote steps ka dher hai — koi ek step bhi mushkil nahi, bas ginti mein zyada hain. Aap ka asal kaam 4 cheezein hain jo AI kabhi nahi karega: **(1)** accounts/keys/approvals, **(2)** quality judgment (articles parhna, model chunna, prompts tune karna), **(3)** real testing apne phone/site pe, **(4)** clients dhoondhna. Baqi sab Claude Code ka kaam hai. Har hafte ke end pe apne aap se ek hi sawal poochho: *"kya is hafte ka checkpoint pass hua?"* — haan to aage barho, nahi to usi step pe ruk ke fix karo. All the best! 🍀